

Using metrics to improve software maintenance

R Hall and S Lineham

Software maintenance is a huge cost within any large software-rich business and can consume a significant proportion of support budgets. The quality of software delivered into BT's Systems Integration unit has a major impact not only on the cost and time to integrate, but also on the long-term support of the system. Systems Integration is sponsoring research into ways of assessing software to help with early identification of potential maintenance problem areas and pointing to ways of improving the quality of systems. This paper describes the work that is being done in this area, and focuses on practical applications for assessing code quality, as well as covering some of the ongoing research into software maintenance.

1. Introduction

Software maintenance can be defined in many ways and covers the spectrum from fixing bugs to major enhancements. The following is the ANSI/IEEE definition:

'Software maintenance is the modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified environment.'

However it is defined, the cost of changing systems for whatever reason is a major expense for any software-rich company. The costs of maintenance impact on BT's Systems Integration (SI), as much as anywhere else. Not simply the cost of changes to software to enable the integration process, but also the ongoing support costs for the life of that software which could involve several iterations of integration as systems become ever more complex and interdependent. The cost is not restricted to the resources required to perform the maintenance. In a highly competitive market the speed with which working systems can be delivered can be crucial in winning and keeping market share, and any delays caused by difficulties in changing software are potentially critical to business success. To meet this challenge it is essential to strive for software that can be easily maintained both to drive down the direct costs of change and the speed at which it can be evolved to help protect market share. With the backing of BT's Corporate Research programme, SI has been looking at ways in which maintenance can be improved, and specifically, whether there are ways to assess the code being delivered into SI either for integration or maintenance to give some indication of potential problems, and, in particular, to see if it would be possible to:

- provide an assessment as to the quality of code,
- provide a wide range of measures, not just process and size but also data content, testability, information flow and others,
- enable an assessment of the amount of change that has occurred over successive releases of a software system,
- allow the change in quality of software to be monitored over successive releases of a software system,
- enable testing effort to be estimated and targeted,
- enable the identification of code that is a good candidate for quality improvement/re-engineering,
- enable an assessment of the cost and risk of changing particular source code,
- provide a basis for reviewing code changes.

The first avenue explored was to see if a consistent set of independent measurements could be identified which could make meaningful predictions about the future maintenance liability of the code. Independence of the measurements is a major consideration. There are two issues with collection of metrics. The first is simple — people are better employed actually delivering systems than counting what they are delivering. When a team is up against tight deadlines, which is the normal situation, there is little time available to collect measurements and the quality of what data is collected manually may well suffer as a result, and imperfect metrics are unlikely to deliver meaningful results. The second problem is subjectivity. Whatever metric you choose to collect is likely to have

some degree of interpretation, and if there is no absolute standard for what the metrics are, then again there is some doubt over the value of any results. A good example would be collecting fault metrics — exactly what do you categorise as a ‘fault’?

There are an enormous number of recognised metrics and some of these would be extremely difficult to collect manually — especially on large systems, although there are a number of commercially available tools that can collect them. However, what use can a systems integrator (or a software engineer) make of a large set of numbers? Specifically, what can the measurements tell us about the maintenance liability of the system that is being measured? It was the investigation of just what could be learned that led to the development of a prototype tool that has become known as the Code Measurement Toolkit.

2. Code Measurement Toolkit

The Code Measurement Toolkit (CMT) is an integrated environment for the code analysis and maintainability assessment of C and COBOL code. These languages were chosen for the research prototype as they account for the large majority of BT systems. CMT was developed as part of a joint partnership between Systems Integration and the Systems Research Division. CMT can be used to:

- predict the likelihood of future change to a file,
- use two maintainability indices to assess the maintainability of code,
- use a comprehensive set of industry-leading software metrics including data-flow measures to assess the complexity, testability and readability of code,
- export a set of metrics for use by the Code Monitor successive release monitoring tool.

2.1 Rationale for CMT

The ideas behind CMT are derived from over four years of research. An extensive data collection and analysis exercise was carried out, encompassing five large software projects within BT, in order to identify the code metrics which were most strongly related to the occurrence of software faults and code changes. The research indicated that with around 70 — 80% accuracy it should be possible to predict, from analysis of the code metrics, which source code files in a system are likely to require changing in the future.

2.2 Details of CMT

CMT was developed as a prototype tool aimed at exploiting the results of this research. CMT consists of three main components:

- code measurement,
- file change prediction capability,
- graphical user interface.

Code measurement

CMT utilises the code parsing and metrics tool X-RAY, developed by South Bank University, as the code-parsing tool. Further code measures are calculated from X-RAY’s output, using the Qualms tool and other internally developed algorithms. In total 17 system level, 58 source file level and 44 function level measures are made (function in this context is taken as being synonymous with perform and procedure). These metrics include measurements of the following aspects of source code:

- size,
- processing complexity,
- data complexity,
- information flow between source code files and between functions,
- testability,
- maintainability.

Some of the file level measures appear in a number of statistically derived forms — max, mean, weighted mean, and average density (the average density of a metric for a portion of source code is calculated by dividing the metric value by the number of non-comment, non-blank lines of code).

The range of measures made is very broad which enables a very good understanding of various attributes of the code to be obtained. Current commercially available tools do not provide the ability to make such a broad range of measures.

File change prediction capability

The prediction of the likelihood of a file changing in the future is based on the analysis of the measures produced by CMT. Two expert systems are used to independently analyse the measurements — a neural net and a rules-based expert system. The results from the two expert systems are then compared to allow a judgement to be made.

To enable the two expert systems to analyse the code measurements they first have to be trained on a set of source code that has a known change history. The source code is measured and the measurements and change history are analysed by a data-mining engine to extract the rules for the rules-based expert system and by the neural network retraining tool. Full details of the retraining process and

other architectural aspects of CMT can be found in Heath [1]. The code and change history of release 32 of four sub-systems, 'DI', 'OC', 'OS' and 'RH' from BT's customer service system (CSS), was used for training CMT. The sub-systems were of varying size and contained in total approximately 1 million lines of source code. The file change predictions were validated by checking the predictions of change against the amount of change that had already occurred to the code used for training. The predictions were found to be >90% accurate. The prediction accuracy was also tested with a C source code system, of some 90 source files, for which the source code for the first and each subsequent release was available. The source code for the first release was measured. The predictions of change showed all the files as 'Least Likely to Change'. This was compared with the number of times that each file actually had been changed. The source code was of generally good quality and the predictions made by CMT were correct. Once the training is complete CMT can then be used to make change predictions of other source code.

Graphical user interface

The graphical user interface was developed using the TeleUSE GUI builder.

2.3 Using CMT

CMT can be used to process all the source code files in a system or individual files. Once measurements of source code have been made they can be saved for viewing and further analysis at a later date. Full user instructions can be found in Sharples [2].

The file change prediction results for the set of source files measured are presented in the 'File Change Predictions' window. The window, an example of which is given in Fig 1, is composed of three scrolled lists (scroll bars will only be displayed if required):

- the topmost list contains all the files that have been given a **fail** rating, i.e. those files that are most likely to change in the future,
- the middle list contains all the files that have not been given a rating — this occurs if the two expert systems give conflicting results for a file, e.g. the neural net passes a file which the rules-based expert system fails,
- the bottom list contains all of the files that have been given the rating of a **pass**, i.e. those files that are least likely to change in the future.

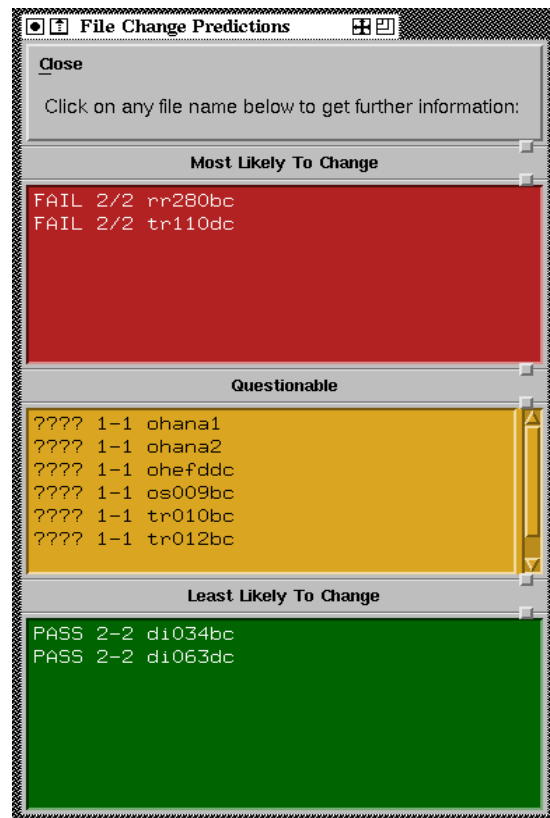


Fig 1 The 'File Change Predictions' window.

A list of the expert system rules that were used to make a judgement about a file can be obtained and, as a consequence, improvements to the code can be suggested.

Code maintainability is measured using the Hewlett Packard Maintainability Index. This index is an empirically determined, industrially validated metric, and is derived from a subset of the CMT metrics. The HP measure has three possible scoring zones. The results are presented in a window of similar format to the file change predictions (an example is given in Fig 2):

- the 'poor maintainability' list shows files scoring less than 65 — these files are very likely to be difficult to maintain in the future,
- the 'reasonable maintainability' list shows files scoring between 65 and 85 — these files are likely to be maintainable, though may have some difficult areas,
- the 'good maintainability' list shows files scoring over 85 — these files are likely to be easy to maintain.

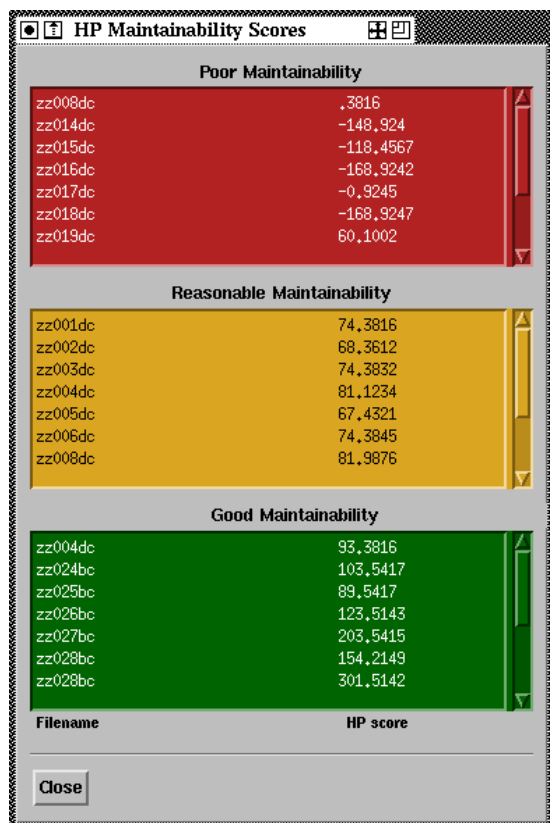


Fig 2 HP maintainability index.

The prediction of likelihood of change can be used together with the maintainability measure to identify code that would be a good candidate for improvement. For example, if a particular source file is likely to change a lot in the future and it is unmaintainable, then it would be a good investment to improve the quality of the code so that future changes are easier to perform and involve less risk.

As well as viewing file change predictions and the maintainability measure all the metrics for the code can be viewed, such as those for a particular file or function, or the results for one particular metric, at file or function level, for all the source code measured. Examples of these results are given in Figs 3 — 6.

The buttons on the right (in Fig 3) can be used to view different statistical deviations of certain metrics. This particular metric has no statistical deviations, so all selection buttons are shaded out.

Figure 4 gives the function, metric value and the name of the file in which the function is defined.

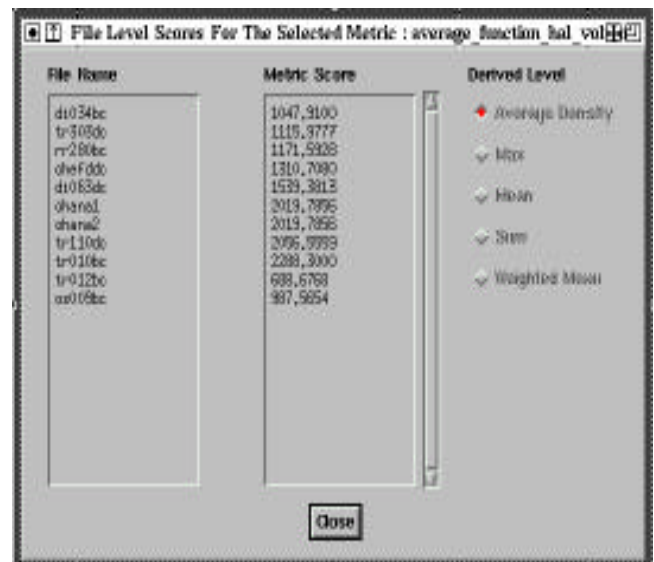


Fig 3 File level results for a particular metric.

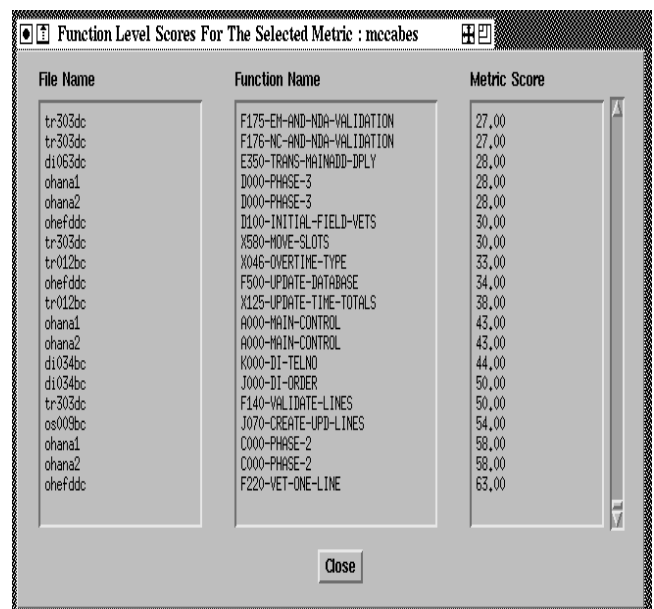


Fig 4 Function level results for a particular metric.

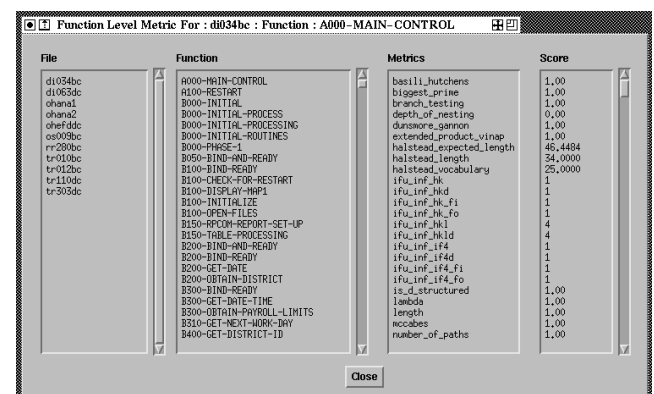


Fig 5 All the measures made for a particular function.

File Name	Function Name	Metric Score
tr303dc	F175-EM-AND-NDA-VALIDATION	27,00
tr303dc	F175-NC-AND-NDA-VALIDATION	27,00
d1063dc	E350-TRANS-MAINADD-DPLY	28,00
ohana1	D000-PHASE-3	28,00
ohana2	D000-PHASE-3	28,00
oheddc	D100-INITIAL-FIELD-VETS	30,00
tr303dc	X580-HOVE-SLOTS	30,00
tr012bc	X046-OVERTIME-TYPE	33,00
oheddc	F500-UPDATE-DATABASE	34,00
tr012bc	X125-UPDATE-TIME-TOTALS	38,00
ohana1	R000-MAIN-CONTROL	43,00
ohana2	R000-MAIN-CONTROL	43,00
d1034bc	K000-DI-TELNO	44,00
d1034bc	J000-DI-ORDER	50,00
tr303dc	F140-VALIDATE-LINES	50,00
os003bc	J070-CREATE-UPD-LINES	54,00
ohana1	C000-PHASE-2	58,00
ohana2	C000-PHASE-2	58,00
oheddc	F220-VET-ONE-LINE	63,00

Fig 6 All the measures made for a particular file.

The measures produced by CMT are stored in a text file so they can also be imported into a spreadsheet for further analysis.

3. Code Monitor

Code Monitor is a tool that provides a means of monitoring:

- the amount and location of code change,
- the change in code quality and maintainability of successive releases of software.

Code Monitor was developed as a prototype tool by Systems Integration.

3.1 Rationale for Code Monitor

Code Monitor was developed following a Systems Integration verification, validation and testing (VV&T) request for some measure of the amount and location of change that had occurred between releases of a software system. This information would then enable better estimates to be made of the VV&T effort required to test the new release, and some indication to be given of the areas in the code where testing effort should be concentrated. Code Monitor could also be used to monitor software developed or maintained by third-party suppliers. Code Monitor measures and reports the change in certain source code metrics for each release of the source code.

The tool was designed on the assumption that a new release of a software system may not include all the source files that comprise the system.

3.2 Details of Code Monitor

Code Monitor uses a subset of the measurements produced by CMT as input. These measurements are loaded

into a database and then the contents of the database are searched to determine whether any previous measures for the source exist. If previous measures are available, the percentage change in each of the measures for each source file is calculated and summed, with some proportional adjustment for source file size. The absolute values are summed for all the source files in the release to give an indication as to the total amount of change that has occurred in the release, this value is called 'total change'.

The changes are also summed taking into account the sign of the change, this total gives some indication as to the type of change that has occurred, i.e. has the software got more or less complicated and bigger or smaller — this value is referred to as 'error proneness' (EP). These values are calculated and displayed for all the source code in each release. An example output is given in Fig 7.

Release	Description	Total Change	E P	Files Changed	Faults Raised	Faults Fixed	Release Date
2	test two	228	184	3	0	0	05/02/96
1.5	test three	29	9	1	0	0	05/02/96
1	test one	0	0	0	0	0	05/02/96

Fig 7 System level metrics display.

A view of all the files in a release (Fig 8) shows the files in order of decreasing value of 'Total Change'. This enables the file that has changed the most to be easily identified.

File Name	Number of Statements	H & K Info Flow	M McCabe	Sum Vmap	Data Oriented	Total Change	E P
compare	401	1907	1.5662	1.4345	1.2632	1280	1280
test	491	1951	1.6562	2.4354	1.2629	135	124
check	390	1591	1.5052	.0534	.1069	70	-19

Fig 8 View of all the files in a release.

All the functions in a file can also be viewed again in descending order of 'Total Change', see Fig 9.

Procedure	Number of Statements	H & K Info Flow	McCabe	Sum Vinaps	Total Change	Error Proneness
BBA-LINK	0198	3	.45	.25	69	-69
BB-PHASE-B	199.0098	505774	28.98	264.98	10	-10
B-MAIN-ROUTINES	20	452	10	59	0	0
BC-INPUT-DATA	24	14874	4	6	0	0
BCA-VALIDATE-UPDA	29	15554	5	12	0	0
BCAA-MODIFY	12	1442	2.98	6	0	0
BCAB-CEASE-MANDA	27	122502	4	6	0	0
C-FINAL-ROUTINES	8	8	4	5	0	0
THE-CORE	6	6	5	9	0	0
X-ERROR-ROUTINE	7	82	3	3	0	0
X-PF-KEY-ERROR	7	182	3	3	0	0

Fig 9 View of all the functions in a file.

The data at system, file and function level can also be viewed graphically (Fig 10). Full details of the use of Code Monitor and the functionality provided can be found in Morton [3]

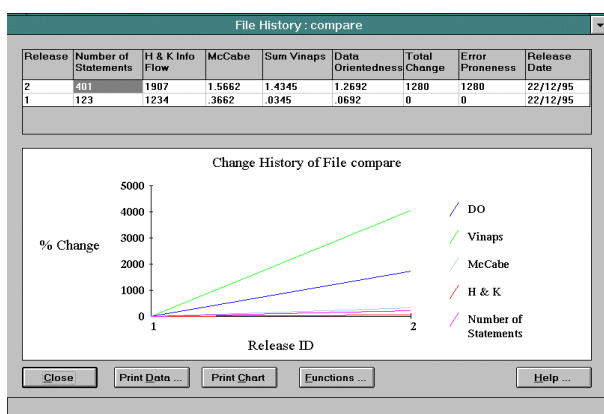


Fig 10 Graphical view of file history.

4. User experience of the tool-set

4.1 Tool evaluation

In order to assess the usefulness of the tool-set, an evaluation exercise was undertaken; this was performed by a CSS development team based in Swindon. The objectives of the trial were:

- to make recommendations regarding the operational use of the tool-set, i.e. CMT and code monitor,

- to provide developers with an objective assessment of the potential uses and value of the Code Measurement Toolkit (CMT), and Code Monitor to CSS,
- to identify issues that exist around the use of the tool-set,
- to provide CSS sub-system managers with initial information regarding the structure and quality of components of their respective sub-systems (to support recommendations for infrastructure improvements),
- to provide suggestions for enhancements to CMT and Code Monitor.

Members of the team were each given access to CMT and Code Monitor. Utilities were developed to assist the use of the tools in a CSS environment. These enabled users to initiate the compilation of sets of CSS source files, thus enabling all the source files in a sub-system to be compiled and copybook expanded source code to be extracted from the job logs produced by the compiler. Full instructions on the use of CMT and Code Monitor in a CSS/MVS environment can be found in Lineham and Morton [4].

For the purposes of the evaluation the source code of four CSS sub-systems, 'DI', 'OC', 'OS' and 'RH', was measured at release 32, 33 and 34 using CMT. The results saved and also loaded into Code Monitor. The development teams were working on developments to the source code of the 'DI' and 'OS' sub-systems for release 35. The source files being changed were measured and the results loaded into code monitor.

Benefits

Overall, the evaluation verdict is 'there would be definite benefits to be realised as a result of using the CMT and Code Monitor in a COBOL environment such as CSS'. The report highlights a number of areas where the tools could prove beneficial, for example:

- better forecasting of test and development effort,
- increased ability to improve performance and infrastructure,
- substantiated quality improvements,
- intuitive judgements supported by hard facts,

- visibility of results provided,
- increased quality control,
- resource utilisation forecasts supported,
- a far greater array of measures provided than available from other tools.

Criticisms

The report makes one major caveat to its introduction — the question of long-term support, an issue that is currently being addressed. There are also a number of constructive suggestions for improvements to the tool-set.

Conclusion

The report concludes that: 'The tool-set would be of great value to the CSS development and testing community and should be introduced', subject to resolution of the long-term support issue. The full findings of the evaluation can be found in Drury [5].

4.2 Other user experience

Other units in BT have used CMT and Code Monitor in assessing other C and COBOL source code and have found the tools very useful in enabling problem areas in code to be quickly identified.

5. Conclusions

CMT and Code Monitor provide functionality that is not currently available in commercial products. The tools are of prototype quality but have been used to process over 4 million lines of source code. The information provided by the tools enable informed judgements to be made as to the:

- quality of code,
- risk of changing code,
- testability of code,
- degradation/improvement in code quality over time.

The tool-set is currently being enhanced to enable further automated analysis of code measurements to be made. The rules being developed in the unit when performing code assessments will be included in an analysis tool and embodied in the tool-set.

References

- 1 Heath D: 'CMT Technical Guide', BT Internal Report (1996).
- 2 Sharples J D: 'Code Measurement Toolkit — User Guide', BT Internal Report (1996).
- 3 Morton K: 'User Manual for Code Monitor', BT Internal Report (1996).
- 4 Lineham S and Morton K: 'Measuring code using CMT and Code Monitor', BT Internal Report (1996).
- 5 Drury M: 'Code Measurement Toolkit/Code Monitor package evaluation report', BT Internal Report (1996).



Richard Hall joined BT in 1973 and since 1985 has held a number of managerial positions with responsibility for software development and maintenance, with systems ranging from large support systems to real time applications.

He is currently the resource manager for Call Centre deployment within Systems Integration department, but retains responsibility for the software maintenance of a number of legacy systems including UXD5 and Monarch. Since 1994, he has also directed the BT research into software maintenance techniques on behalf of Systems Integration department and the Corporate Research programme.



Prior to joining BT Steve Lineham worked for the Ministry of Defence developing real time data acquisition and processing systems, for GPT developing Telex control software, and for Plessey Controls Limited developing urban traffic control systems.

Since joining BT he has led and directed research into various aspects of software maintenance. He has also been responsible for a project to provide itemised billing for BT's UXD5 telephone exchanges.

He is currently project manager for several of BT's internal Call Centre projects. He is a member of the British Computer Society, a Chartered Engineer and has attained European Engineer status.